ICT461

HTTP Fundamentals



Methods, status codes, headers, content negotiation, caching, CORS, cookies and HTTP/3

Industry skill: read the network tab like a professional engineer

Mr E Nyirenda

Why HTTP matters in real systems

- Every web action is translated into HTTP requests and responses.
- Poor HTTP choices cause slow pages, broken APIs, duplicate orders and security weaknesses.
- Professional engineers debug with evidence from the network layer, not guesswork.



HTTP is the contract between front end, back end, database and operations teams.

Practical mindset: every lab must show the request, response and reason.

Competencies for this unit

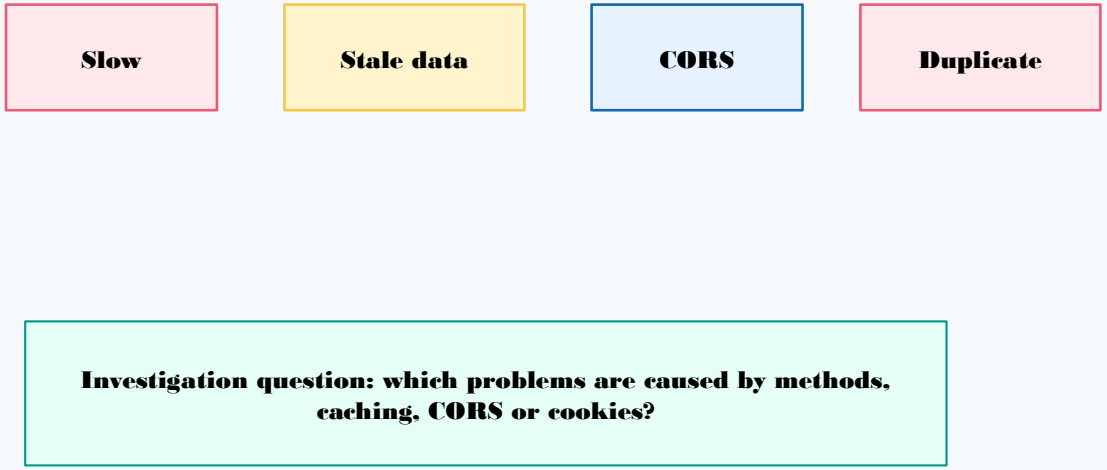
- Explain HTTP request and response structure using real browser evidence.
- Choose appropriate methods and status codes for API operations.
- Apply headers for content negotiation, caching, cookies and security controls.
- Diagnose CORS, cache and cookie problems in a client–server application.
- Describe how HTTP/3 over QUIC improves transport behaviour in modern deployments.

By the end, you should be able to inspect, explain and fix common HTTP behaviour in a web system.



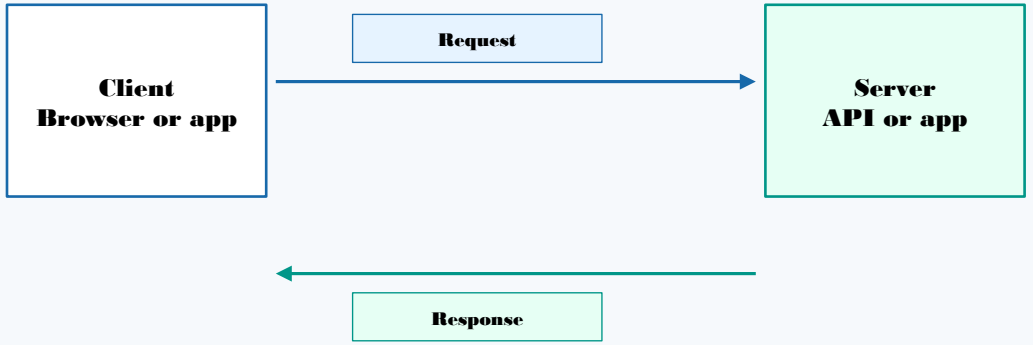
Opening scenario: slow online registration

- A university registration portal becomes slow when thousands of students log in.
- Some users see old timetable data after updates.
- Others receive “blocked by CORS policy” errors from the browser.
- A few students submit the same course twice after refreshing the page.



Mental model: request and response

- A client sends a request describing what it wants.
- A server returns a response describing what happened.
- HTTP is stateless, so every request must carry enough context through headers, cookies or tokens.
- The browser enforces rules such as CORS and cookie restrictions.



Always ask: What was sent? What came back? What did the browser do next?

The parts of an HTTP request

- Start line states the method, path and HTTP version.
- Headers provide metadata about the client, content and preferences.
- Body carries data for operations such as form submission or JSON APIs.
- Query string carries small filtering or selection parameters.

```
POST /api/students?mode=full HTTP/1.1
Host: portal.mu.ac.zm
Accept: application/json
Content-Type: application/json
Cookie: session=abc123

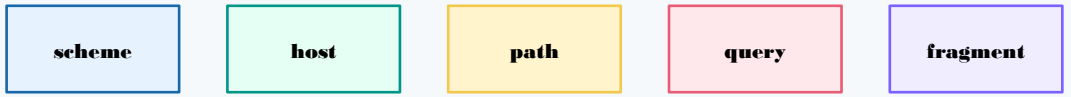
{"course":"ICT461"}
```

Practical habit: label method, URL, headers and body before debugging.

URL anatomy for web engineers

- Scheme tells the browser which protocol to use.
- Host identifies the domain or server.
- Path identifies a resource or route.
- Query parameters filter, sort or search.
- Fragment stays in the browser and is not sent to the server.

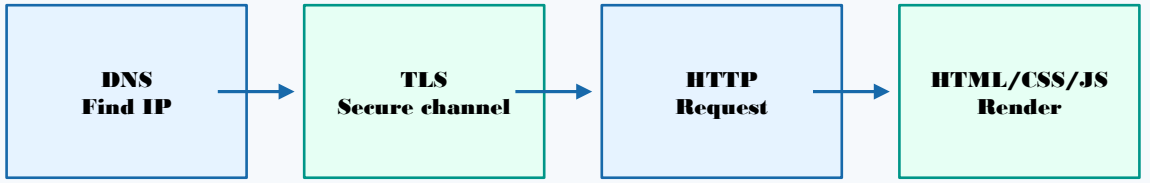
```
https://shop.example.com/products/42?currency=ZMW#reviews
```



Industry example: never put passwords or secrets in query strings.

From typing a URL to seeing a page

- DNS resolves a human-readable name to an IP address.
- The browser negotiates a secure transport connection.
- The request is sent and the server builds a response.
- The browser parses HTML, fetches assets and renders the page.



Network tab evidence often reveals where the delay really occurs.

HTTP methods are not just verbs

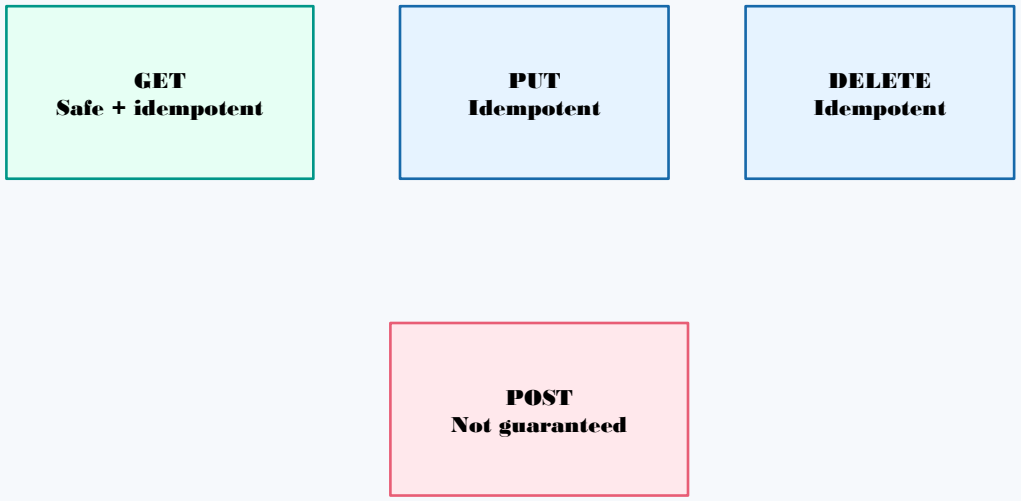
- A method communicates the intention of the operation.
- Correct methods help caching, retries, logging and API documentation.
- Incorrect methods create hidden bugs, especially during refresh, retry and proxy handling.
- Method choice is part of API design, not decoration.



Exam tip: do not use GET for operations that change server state.

Safe and idempotent methods

- Safe means the method is intended only for retrieval.
- Idempotent means repeating the same request has the same intended effect.
- These properties influence caching, automatic retries and fault tolerance.
- Business actions still need duplicate protection at the application level.



Duplicate order prevention is not solved by method choice alone.

GET: retrieval with discipline

- Use GET for reading resources such as product lists, profile views or search results.
- Keep it free of server-side state changes.
- Parameters belong in the URL when they describe filtering, sorting or pagination.
- GET responses can often be cached when headers permit.

```
GET /api/products?category=laptops&page=2 HTTP/1.1
Accept: application/json
```

Good for: loading catalogues, viewing records, searching public content.

Bad for: deleting users, approving payments or resetting passwords.

POST: submit work or create resources

- Use POST when the server performs a process or creates a new resource.
- Typical cases include login, checkout, comments, uploads and payment initiation.
- A successful create commonly returns 201 Created with a Location header.
- Use idempotency keys for critical operations that might be retried.

```
POST /api/orders HTTP/1.1
Content-Type: application/json
Idempotency-Key: 79c2-...

{"items":[{"sku":"A1","qty":2}]}
```



Industry example: payment APIs often protect POST using idempotency keys.

PUT, PATCH and DELETE in practice

- PUT replaces the target resource with the representation provided.
- PATCH changes selected fields without replacing the whole resource.
- DELETE requests removal of a resource or association.
- All destructive actions need authorization, audit logs and careful UX confirmation.

```
PUT /api/users/42
{"name":"Chipo","role":"student"}

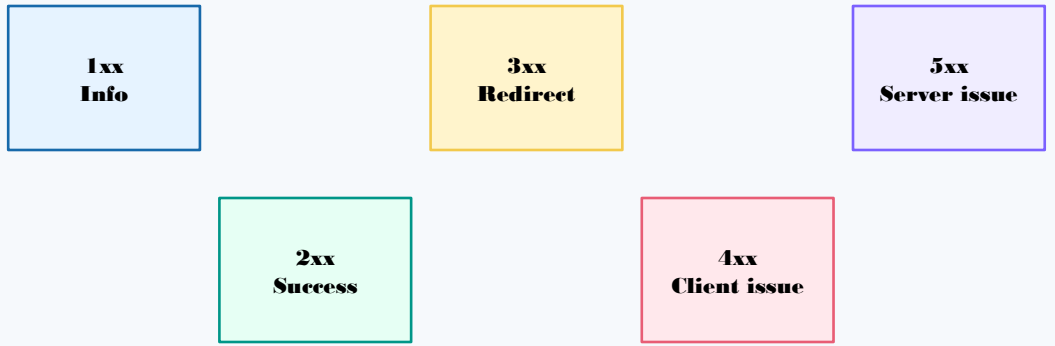
PATCH /api/users/42
{"role":"representative"}

DELETE /api/users/42
```

Real risk: accidental DELETE endpoint exposed without authorization checks.

Status codes: a shared language

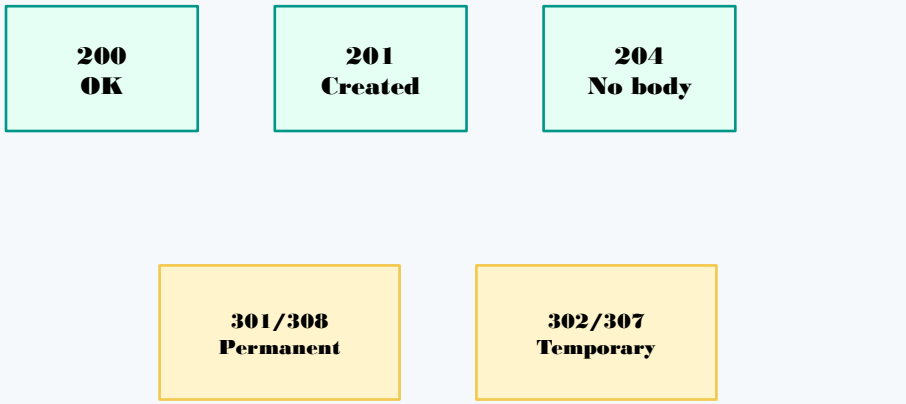
- Status codes summarize the result of a request.
- Clients use them to decide whether to retry, redirect, show validation feedback or stop.
- Good APIs pair status codes with clear response bodies.
- Bad status codes make debugging expensive.



Professional API design: correct status code + useful error message.

Success and redirect codes you will see often

- 200 OK means the request succeeded and a response body is usually returned.
- 201 Created means a new resource was created.
- 204 No Content means success without a response body.
- 301 and 308 indicate permanent redirection.
- 302, 303 and 307 are common temporary redirections.



Redirect chains can slow a site before the page even starts loading.

Client error codes: fix the request or user action

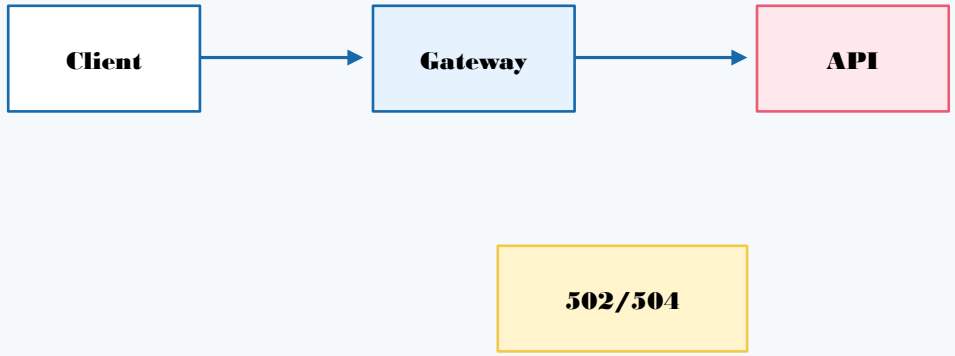
- 400 Bad Request: malformed or invalid request.
- 401 Unauthorized: authentication is missing or invalid.
- 403 Forbidden: user is known but not allowed.
- 404 Not Found: resource was not found.
- 409 Conflict: request conflicts with current state.
- 429 Too Many Requests: rate limit exceeded.



Common mistake: returning 200 OK with an error hidden inside JSON.

Server error codes: fix service reliability

- 500 Internal Server Error: unexpected failure.
- 502 Bad Gateway: upstream service returned a bad response.
- 503 Service Unavailable: service is overloaded or temporarily down.
- 504 Gateway Timeout: upstream service did not respond in time.
- Server errors need logs, correlation IDs and operational response.



Industry practice: give users a simple message, give engineers a traceable error ID.

Headers: metadata that changes behaviour

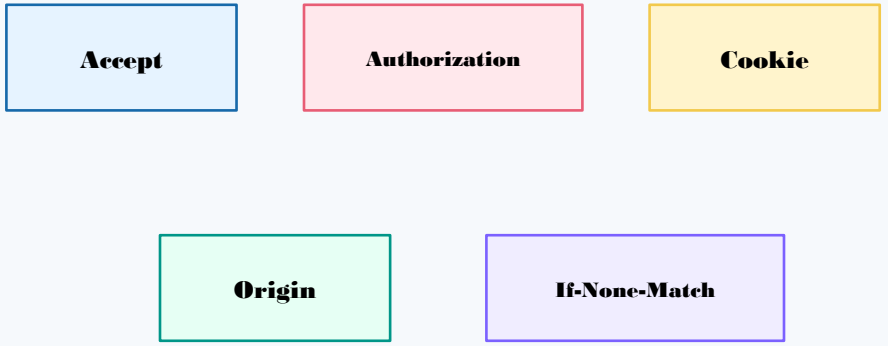
- Headers tell the server or browser how to interpret the message.
- They control format, caching, cookies, compression, security, CORS and tracing.
- Most production HTTP bugs can be understood from a few key headers.
- Headers are case-insensitive, but values must be exact.

```
Accept: application/json
Content-Type: application/json
Cache-Control: max-age=60
Set-Cookie: session=...; HttpOnly
Origin: https://portal.mu.ac.zm
Access-Control-Allow-Origin: ...
```

Debugging shortcut: inspect headers before changing code.

Request headers: what the client is asking for

- Accept asks for a response format such as JSON or HTML.
- Authorization carries credentials when required.
- Cookie sends stored cookie values back to the server.
- Origin is used by the browser for CORS checks.
- If-None-Match helps validate cached content.



Practical example: wrong Accept header can make an API return HTML instead of JSON.

Response headers: how the server answers

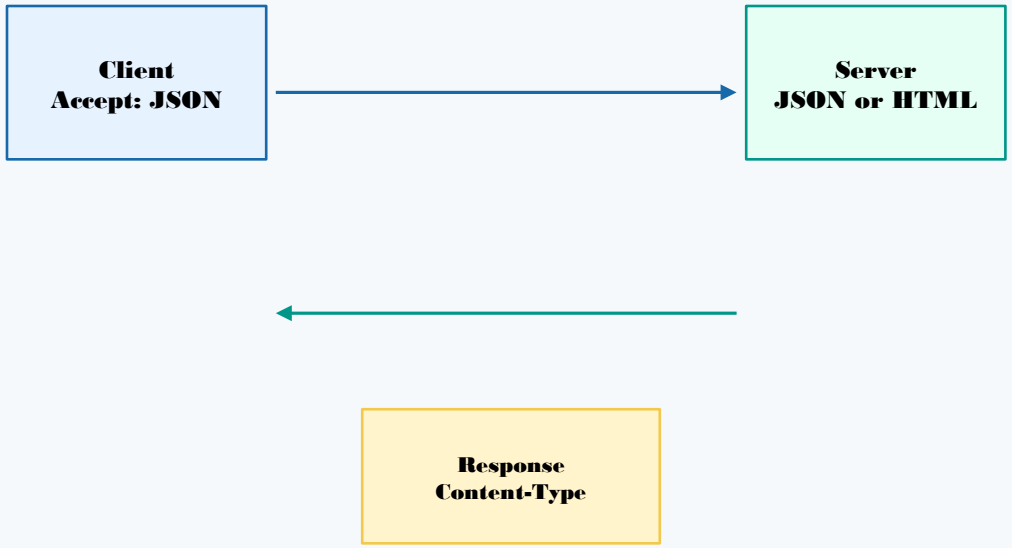
- Content-Type tells the browser how to interpret the response body.
- Cache-Control tells caches and browsers whether to store the response.
- Set-Cookie asks the browser to store cookie values.
- Location points to a created resource or redirect target.
- Security headers reduce the browser attack surface.

```
HTTP/1.1 201 Created
Content-Type: application/json
Location: /api/orders/882
Cache-Control: no-store
Set-Cookie: session=abc; HttpOnly; Secure
```

Question: Which header protects sensitive account pages from being cached?

Content negotiation: same resource, different representation

- The client can ask for JSON, HTML, images or language preferences.
- The server selects a representation based on available content and headers.
- APIs should return predictable formats and clear errors when negotiation fails.
- This supports browsers, mobile apps, integrations and accessibility needs.



Relatable example: the same student record can be rendered as an HTML page or returned as JSON.

Caching: speed with responsibility

- Caching saves copies of responses to avoid repeated work.
- It can happen in the browser, CDN, proxy or application layer.
- Good caching improves performance and reduces server cost.
- Bad caching exposes private data or shows outdated information.



Cache public assets aggressively; avoid caching personal account responses.

Cache-Control in everyday terms

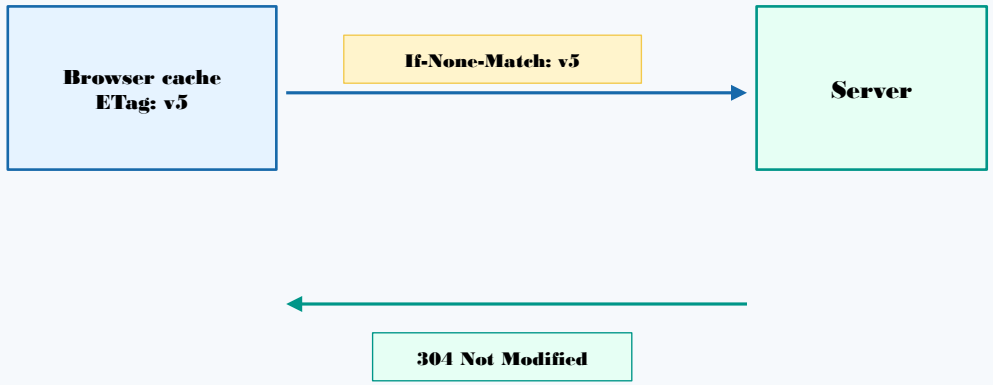
- max-age tells how long a response may be considered fresh.
- no-store tells caches not to store the response.
- private allows browser caching but not shared caches.
- public allows shared caches when appropriate.
- stale-while-revalidate can serve cached content while refreshing in the background.

```
Cache-Control: public, max-age=31536000, immutable
Cache-Control: private, max-age=60
Cache-Control: no-store
```

Rule of thumb: cache fingerprinted assets, not private personal data.

ETag and Last-Modified: validate before downloading

- Validation allows a browser to ask whether cached content is still current.
- If unchanged, the server returns 304 Not Modified with no full body.
- This saves bandwidth and reduces perceived delay.
- ETag is commonly more precise than Last-Modified.



Practical outcome: fast reload without re-downloading unchanged assets.

CORS: browser boundary control

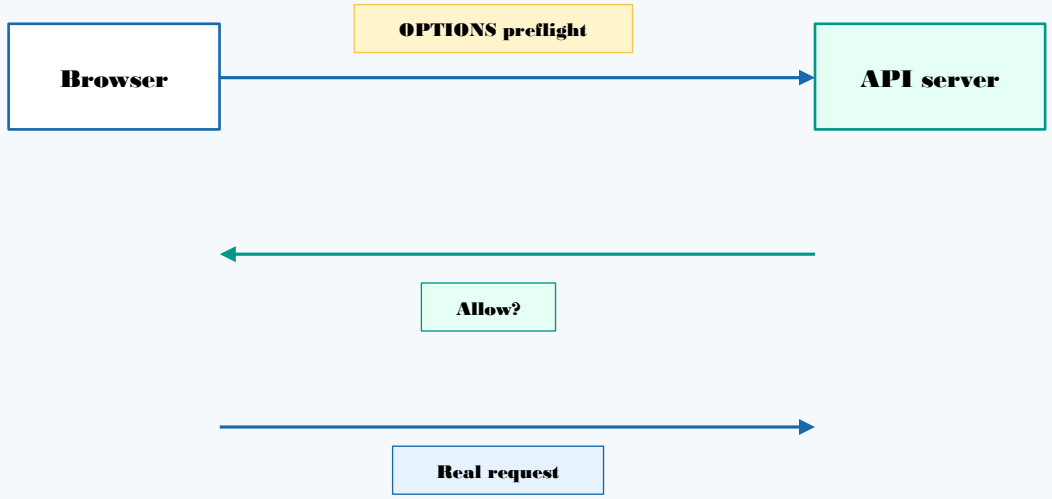
- CORS means Cross-Origin Resource Sharing.
- It controls when browser JavaScript may read responses from another origin.
- An origin is scheme + host + port.
- CORS is enforced by browsers, not by tools like curl or server-to-server jobs.
- It is a browser security mechanism, not a complete API security system.



Different host means different origin, even when both belong to the same organisation.

CORS preflight: OPTIONS before the real request

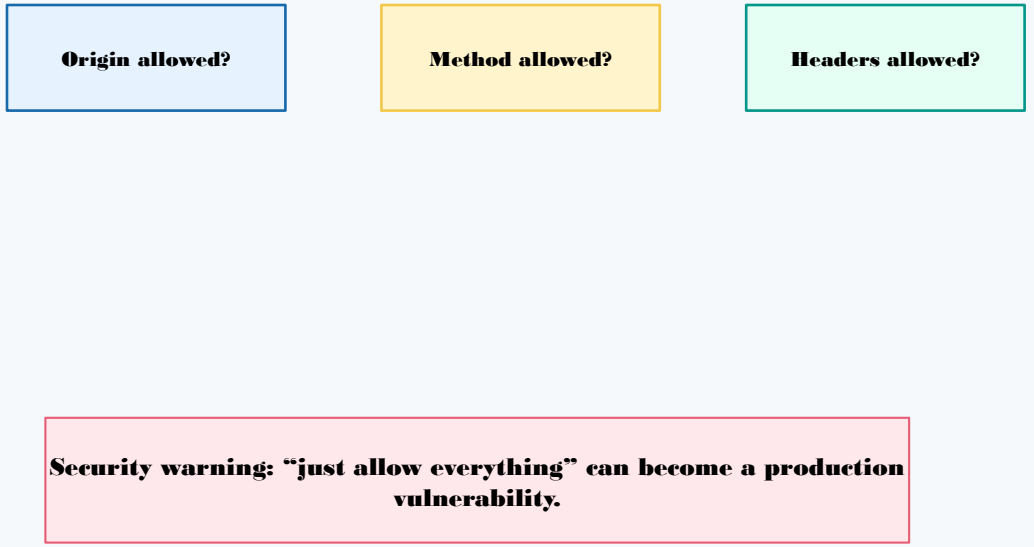
- For some cross-origin requests, the browser sends a preflight request first.
- The preflight asks whether the method and headers are allowed.
- If the server approves, the browser sends the real request.
- If not approved, the browser blocks JavaScript from reading the response.



Debugging step: inspect the OPTIONS request, not only the failed POST.

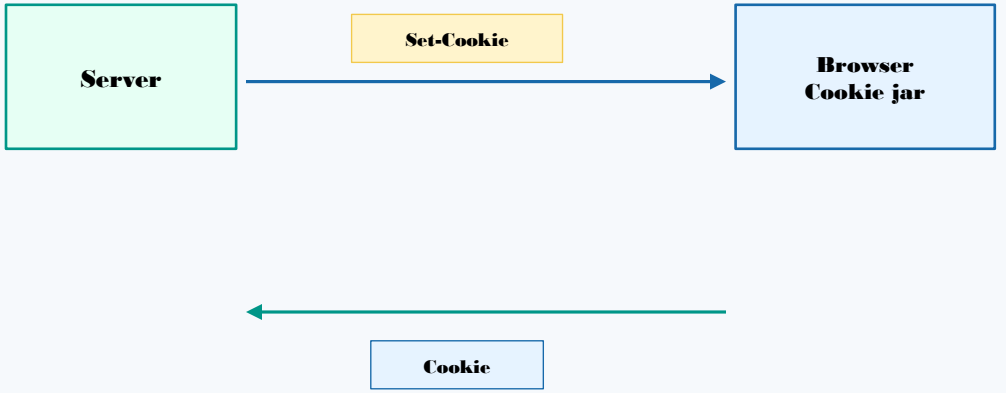
Common CORS mistakes

- Allowing * while also using credentials.
- Forgetting to include the Authorization header in allowed headers.
- Configuring the API correctly but calling the wrong URL or port.
- Solving CORS by disabling browser security during development.
- Assuming CORS replaces authentication and authorization.



Cookies: small state carried by the browser

- HTTP is stateless, but cookies let the browser attach stored values to requests.
- Servers commonly use cookies for sessions, preferences and CSRF protection.
- Cookies are scoped by domain, path, expiry and security attributes.
- Sensitive cookies must be protected against theft and unintended cross-site sending.



Do not store passwords in cookies. Store a random session identifier instead.

Cookie attributes every web developer should know

- HttpOnly prevents JavaScript from reading the cookie.
- Secure sends the cookie only over HTTPS.
- SameSite reduces cross-site request risks.
- Max-Age or Expires controls lifetime.
- Domain and Path control where the cookie is sent.

```
Set-Cookie: sid=abc123; HttpOnly; Secure; SameSite=Lax; Path=/; Max-Age=3600
```



Practical lab: compare cookie behaviour on localhost HTTP and HTTPS.

Sessions, cookies and tokens: choose carefully

- Cookie-based sessions keep server-side session state and send a session ID.
- Bearer tokens are often used for APIs but require careful storage and expiry handling.
- Browser apps must consider XSS, CSRF, token leakage and logout behaviour.
- Authentication is not complete without authorization checks on every sensitive operation.

Cookie session
Server remembers

Bearer token
Client presents

Engineering decision: "Where is state stored, how does it expire, and how is it revoked?"

Security headers: small lines with big impact

- Strict-Transport-Security encourages HTTPS-only access after first trust.
- Content-Security-Policy limits where scripts, styles and other resources may load from.
- X-Content-Type-Options reduces MIME sniffing surprises.
- Referrer-Policy controls how much URL information is shared with other sites.
- Security headers complement secure coding, they do not replace it.

```
Strict-Transport-Security: max-age=31536000
Content-Security-Policy: default-src 'self'
X-Content-Type-Options: nosniff
Referrer-Policy: strict-origin-when-cross-origin
```

Activity: use browser DevTools to confirm whether a response has these headers.

HTTP/2 and HTTP/3: why protocols evolved

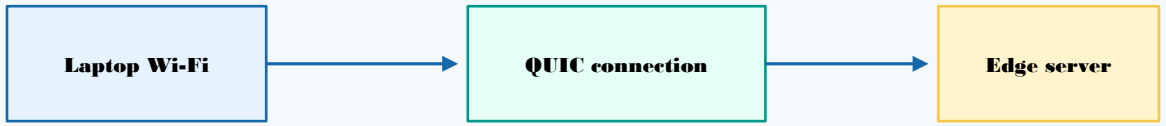
- Modern pages require many assets, API calls and third-party resources.
- HTTP/2 introduced multiplexing over one connection to reduce queueing.
- HTTP/3 runs over QUIC, which uses UDP and improves connection setup and loss handling.
- The developer still designs URLs, headers, caching and APIs carefully.



Key point: newer protocols help transport, but cannot fix a badly designed API.

HTTP/3 over QUIC: practical mental model

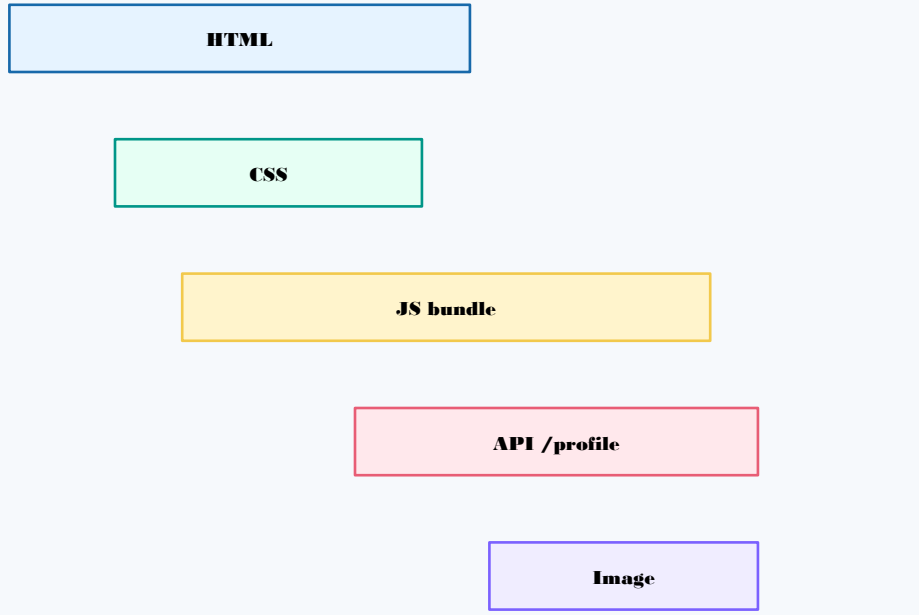
- QUIC combines transport and security handshakes to reduce delay.
- Multiplexed streams avoid one lost packet blocking all streams in the same way as older transport behaviour.
- Connection migration helps when a device changes networks.
- CDNs and edge networks often make HTTP/3 available without changing application code.



Relatable example: moving from campus Wi-Fi to mobile data should not always break the connection.

Performance view: read the waterfall

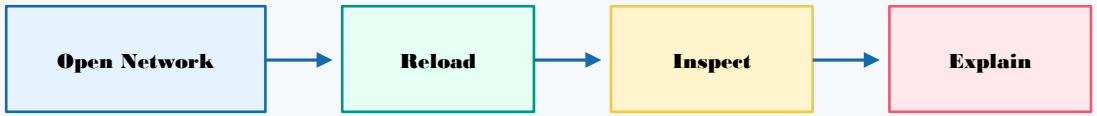
- The waterfall shows when each request starts, waits and completes.
- Look for long DNS, connection, waiting and download phases.
- Repeated uncached assets can reveal missing cache headers.
- Redirects and slow API calls often dominate perceived performance.



Activity: identify one request you can remove, cache or defer.

Debugging workflow in DevTools Network

- Reload with the Network tab open.
- Filter by Fetch/XHR, document, CSS, JS or images.
- Check method, URL, status code, timing and headers.
- Read the response body for validation or error details.
- Copy request as cURL when you need to reproduce outside the browser.



Evidence beats guessing: capture a screenshot of the exact failing request.

Mini lab: build a simple HTTP inspector

- Create a small Node.js or Express endpoint that echoes method, path, headers and body.
- Call it from a browser form and from fetch().
- Change method and headers and observe the difference in Network tab.
- Submit one screenshot and a short explanation of three observed headers.

```
app.all('/inspect', (req, res) => {
  res.json({
    method: req.method,
    path: req.path,
    headers: req.headers,
    body: req.body
  });
});
```

Competency evidence: screenshot + explanation, not just working code.

Practical exercise: design a student API

- Design endpoints for viewing courses, enrolling, dropping and checking status.
- Choose the correct HTTP method for each endpoint.
- Choose a success and failure status code for each operation.
- Write one request and one response example.
- Explain which endpoints can be cached and which must not be cached.



Challenge: prevent duplicate enrolment when the user double-clicks submit.

Industry case: e-commerce checkout

- GET loads basket and delivery options.
- POST creates payment intent or order.
- 409 Conflict can signal stock changed during checkout.
- 303 See Other can redirect to the receipt page after a successful POST.
- no-store protects sensitive checkout responses from caching.



Why it matters: one wrong retry or cache rule can affect money and trust.

Team activity: HTTP audit of a real site

- In groups of four, choose one public website or your own project.
- Open DevTools Network and reload the home page.
- Record five requests and classify method, status, content type and cache behaviour.
- Identify one improvement for performance, security or reliability.
- Present a three-minute evidence-based explanation.



No unsupported claims. Show the request and response that support your point.

Unit 2 competency checklist and assignment

- I can identify method, URL, status code, headers and body in DevTools.
- I can choose suitable methods and status codes for a small API.
- I can explain caching choices and detect stale-content risks.
- I can diagnose a basic CORS failure.
- I can describe cookie attributes and why Secure, HttpOnly and SameSite matter.
- I can explain what HTTP/3 changes at a high level.

Assignment evidence

- Design a six-endpoint student registration API.
- Include method, route, request body, success code and two failure codes.
- Add cache and cookie notes where relevant.
- Submit one Network tab screenshot showing a real request.
- Maximum two pages plus screenshot.

Next lecture: Authoring for the Web